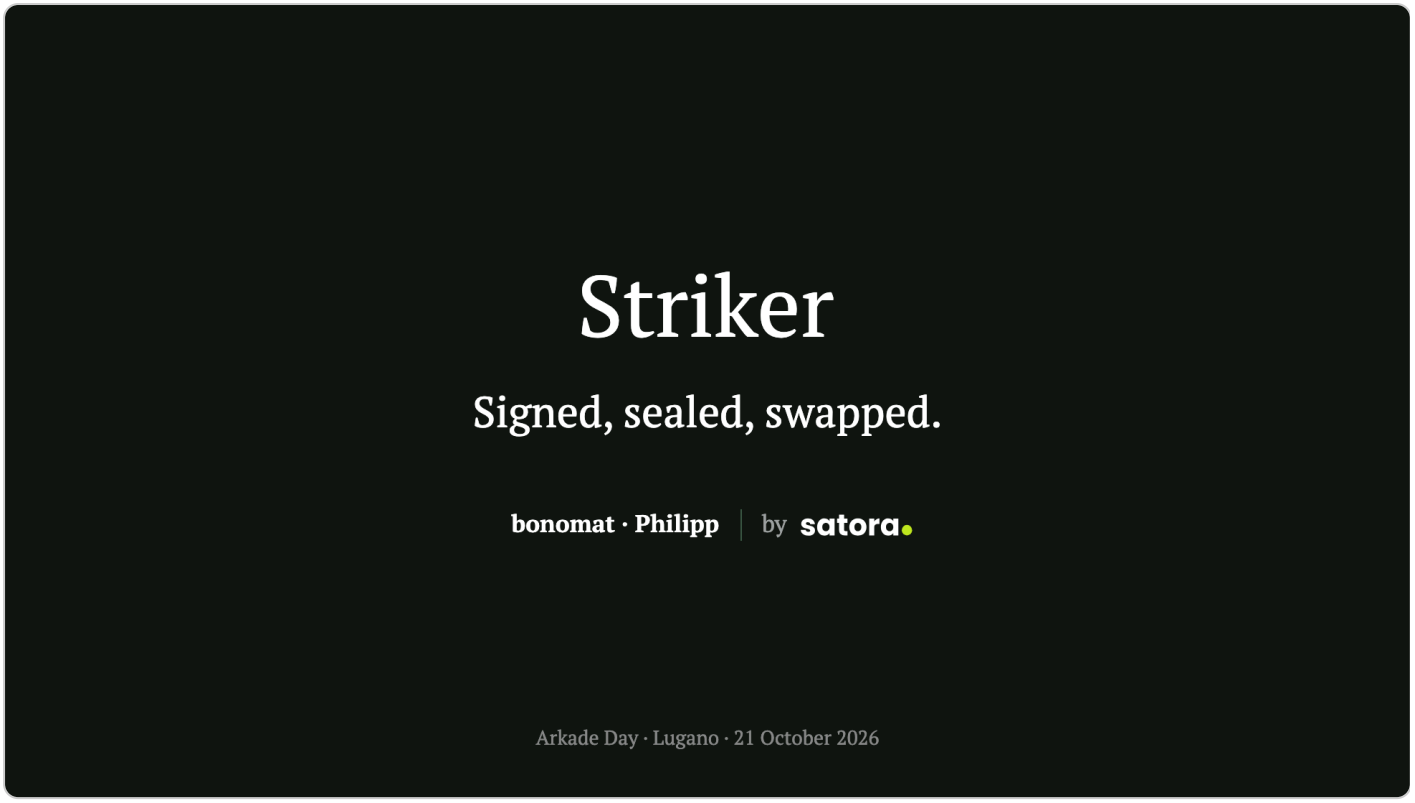


00:00–00:30 · 30 seconds

Hi all



00:30–00:35 · 5 seconds

Trade like a CEX. Keep your keys. That's the bar. Let's start with the price.



00:35–01:30 · 55 seconds

Have you ever pressed swap and received less than the number you first saw?

[Pause.]

For me, a good quote has three properties.

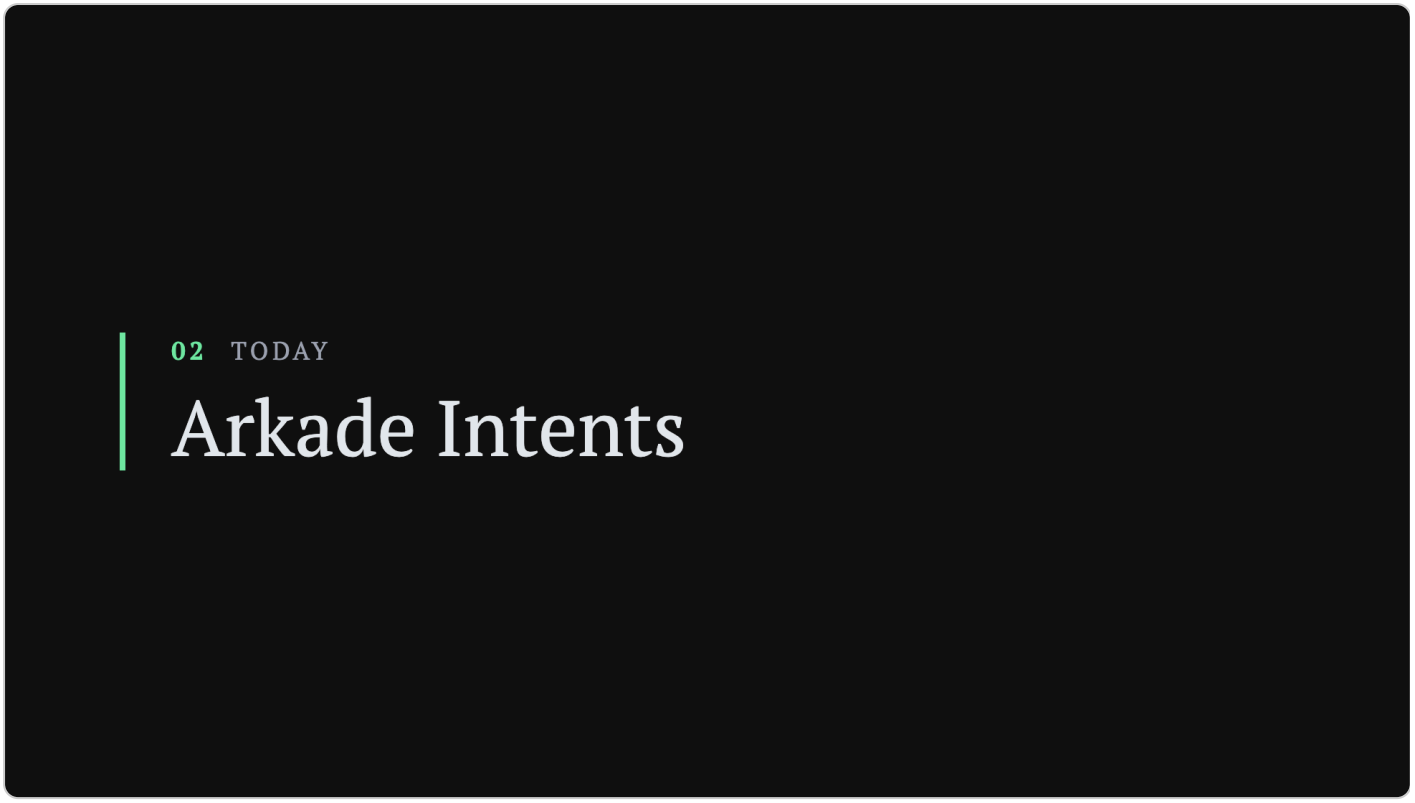
It's fresh when I ask. It reflects the market now, rather than a price my phone downloaded a while ago.

It's firm when I accept. I can see exactly what I'm paying and what I'm receiving.

And the rules are the same whether I'm trading from a fast server or a phone on conference Wi-Fi.

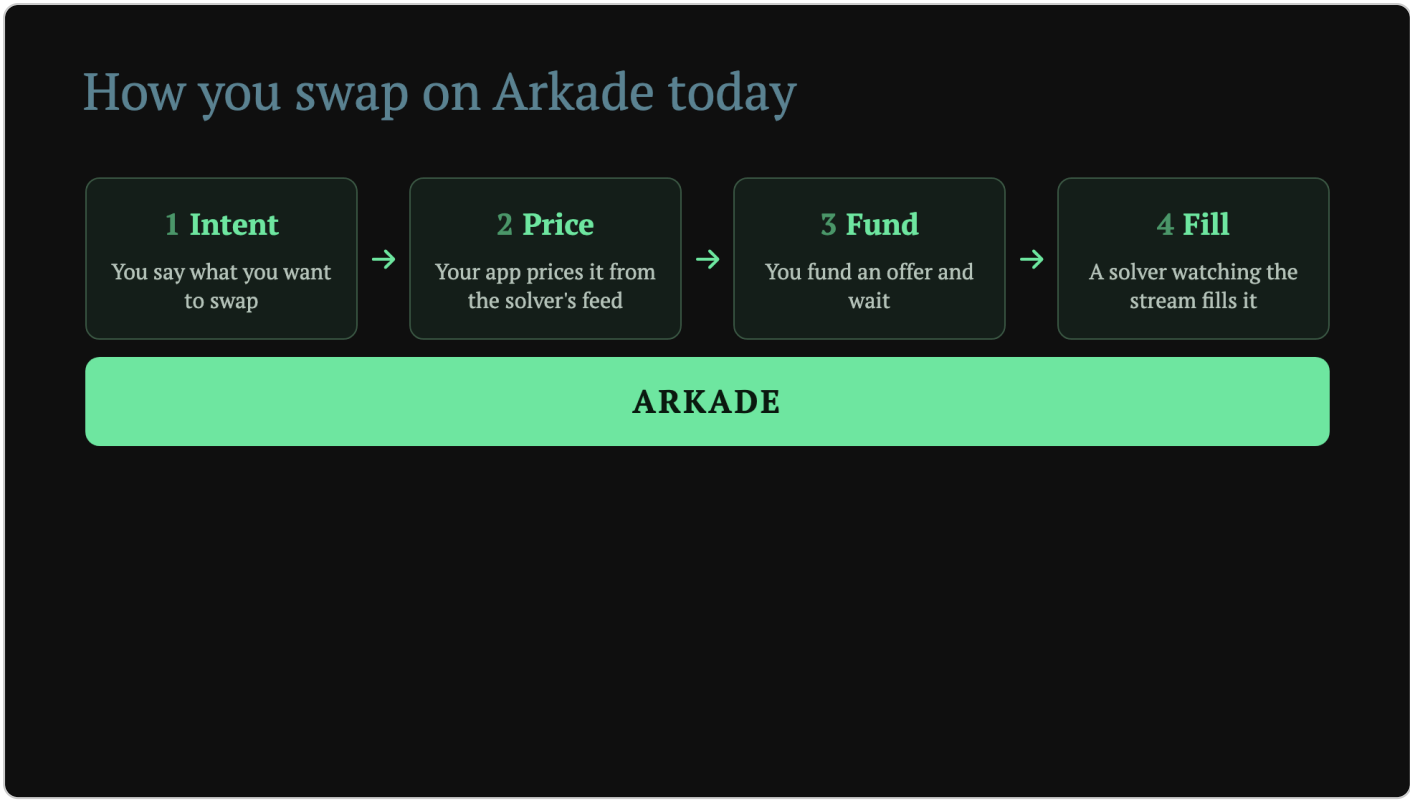
That doesn't mean every user gets an identical price forever. It means an accepted quote has explicit terms that don't change underneath them.

Those are the properties we're exploring.



01:30–01:35 · 5 seconds

First, let's look at the foundation: Arkade Intents.



01:35–02:45 · 70 seconds

Everything I'm showing stands on Arkade. Ark Labs built the settlement environment and the programming tools that make this possible.

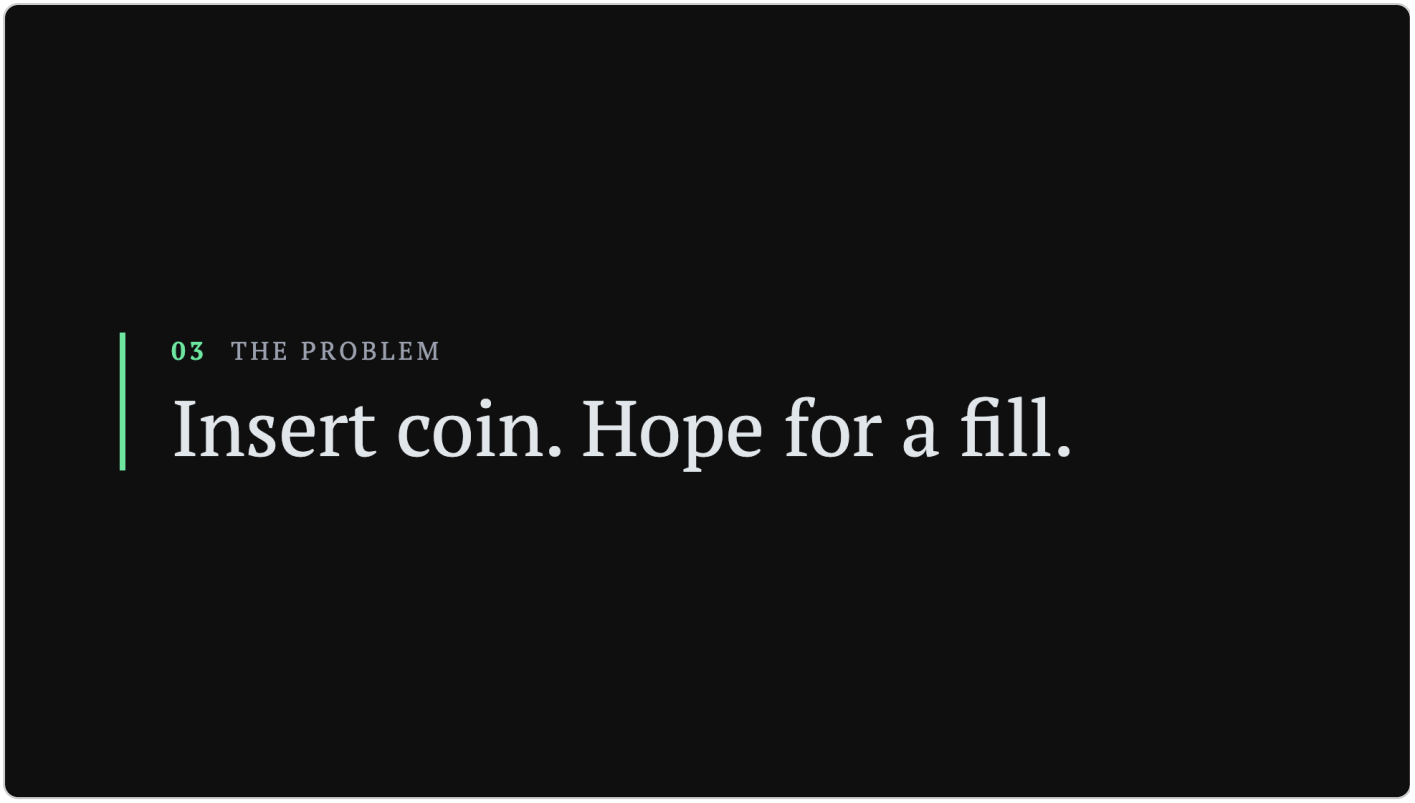
In the standing-offer spot flow we're comparing here, a user chooses a market, derives a price from a feed, and puts an offer on Arkade. A solver can observe that offer and fill it.

That's powerful: the user doesn't have to stay online to negotiate every step, and settlement is atomic. A solver cannot take the payment without satisfying the offer.

There are also request-for-quote designs in the wider Intents ecosystem. So this talk is not a claim that Arkade has never had quotes.

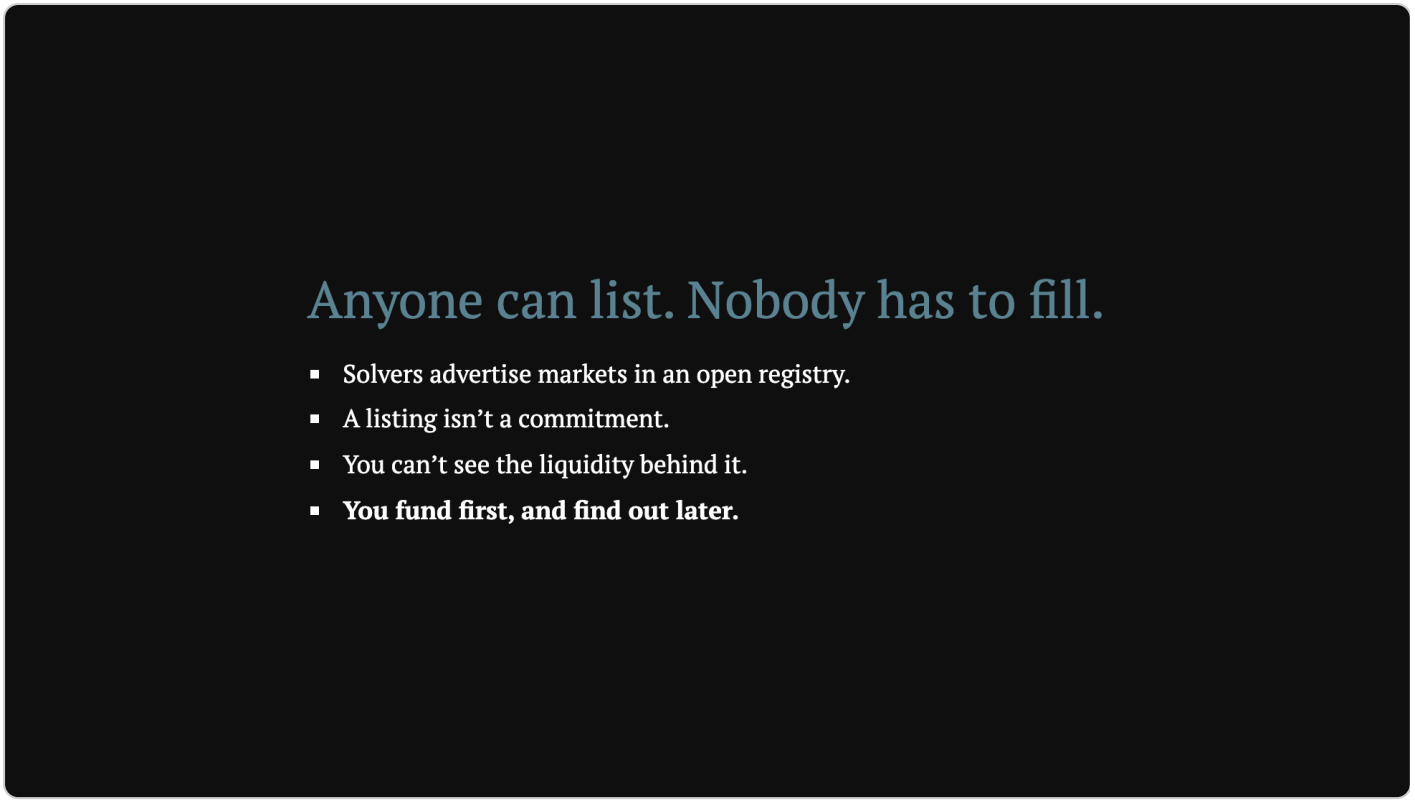
Our narrower question is: for a swap entirely within Arkade, what changes if the liquidity provider signs the price first, against coins already held in a covenant?

That's the experiment Striker adds.



02:45–02:50 · 5 seconds

Insert coin. Hope for a fill. There's a small arcade joke here.



02:50–03:40 · 50 seconds

An open registry is a good thing. Anyone should be able to advertise a market and compete for flow.

But a listing and an executable commitment are different things.

Seeing a market advertised doesn't tell me that someone has reserved the liquidity, or that a particular price will be filled.

In the standing-offer flow, I commit my coins first. Then I discover whether a solver wants the trade.

If nobody fills, I can cancel. My point isn't that the coins are lost. It's the order of commitment: the user acts before the counterparty commits back.



03:40–04:30 · 50 seconds

The next question is who turns a moving market into an executable price.

In this client-priced flow, the wallet reads a feed, calculates the offer and leaves enough room for a solver to accept it.

But the wallet and solver won't read the market at precisely the same instant. Network delay and market movement create a gap.

A safety cushion helps the offer fill, but the user pays for that uncertainty. Too little room, and the offer may wait. More room, and the user gives away more value.

I'd rather ask the counterparty: what price will you commit to right now?



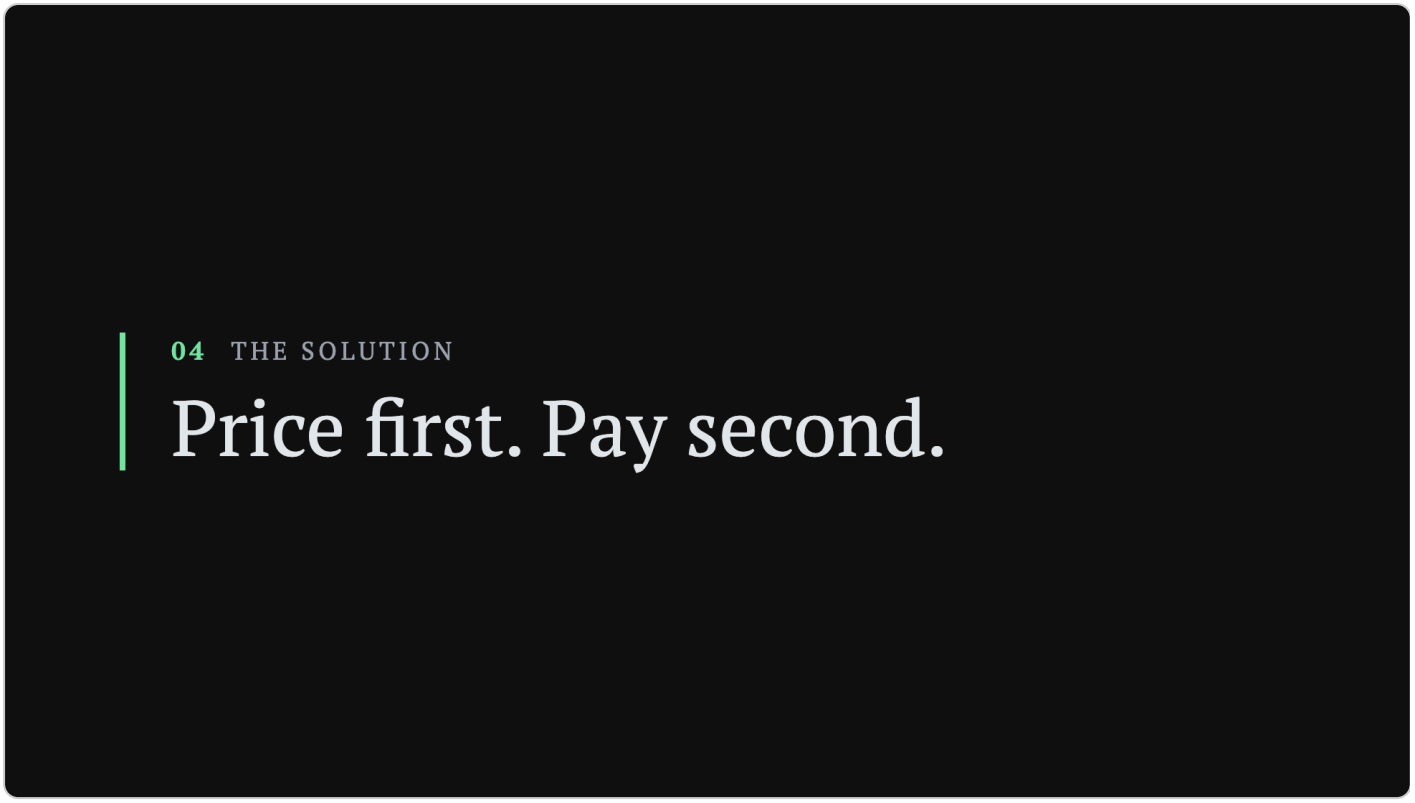
04:30–05:00 · 30 seconds

There's also an operational question on the liquidity side.

A straightforward automated solver keeps spending keys online so it can fill trades. If that machine is compromised, its inventory may be exposed.

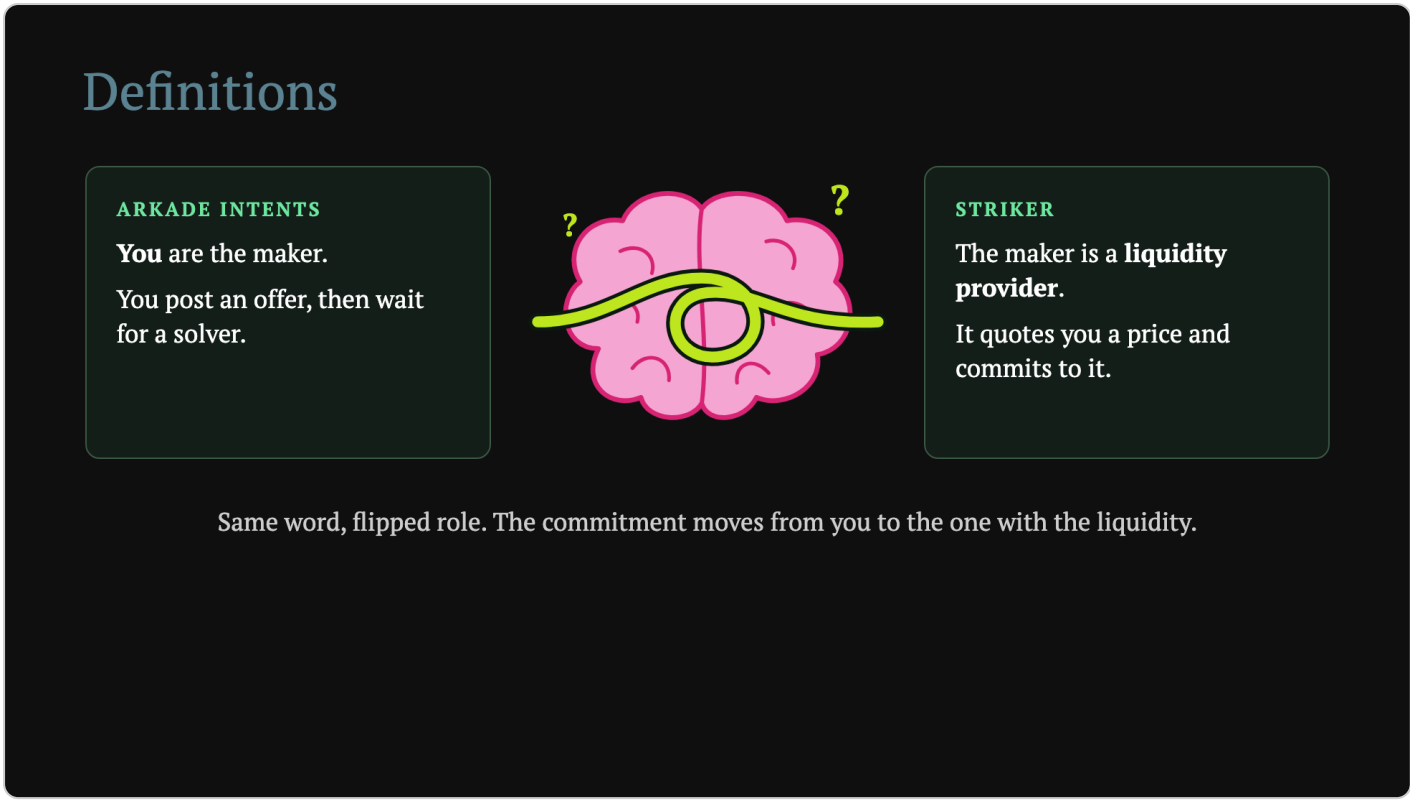
That suggests another design question: can we separate the authority to quote a trade from the authority to withdraw the pool?

Keep that question in mind. We'll come back to it.



05:00–05:05 · 5 seconds

Price first. Pay second. This time, the liquidity provider commits before you do.



05:05–05:35 · 30 seconds

One vocabulary warning, because this tied my brain in a knot.

In the standing-offer convention, you are the maker: you post an offer and wait for someone to take it.

In Striker, I'm using the market-making convention. The maker is the liquidity provider who answers your request with a signed price.

Same word, different role. For the rest of this talk, maker means the liquidity provider.



05:35–06:50 · 75 seconds

[Click: request.]

Your wallet asks Striker for a price. Striker asks its configured makers and selects an executable signed offer.

[Click: signed quote.]

The maker's answer specifies the amounts, the pool and an expiry. The wallet verifies those terms before asking you to accept.

The maker has already deposited liquidity. Its quote is an authorization that can be used to settle a trade against that pool, subject to the covenant's checks.

[Click: authorize and settle.]

Your wallet signs its side of the deal. Striker builds the transaction and submits it on Arkade.

The maker doesn't have to come back online to sign that transaction. Its quote already carries the authorization.

There are still limits: the quote must not have expired, sufficient liquidity must remain, and the pool must not have been cancelled.

[Click: closing line.]

So the change is the order of commitment. First I receive a signed price. Then I authorize my payment.



06:50–07:40 · 50 seconds

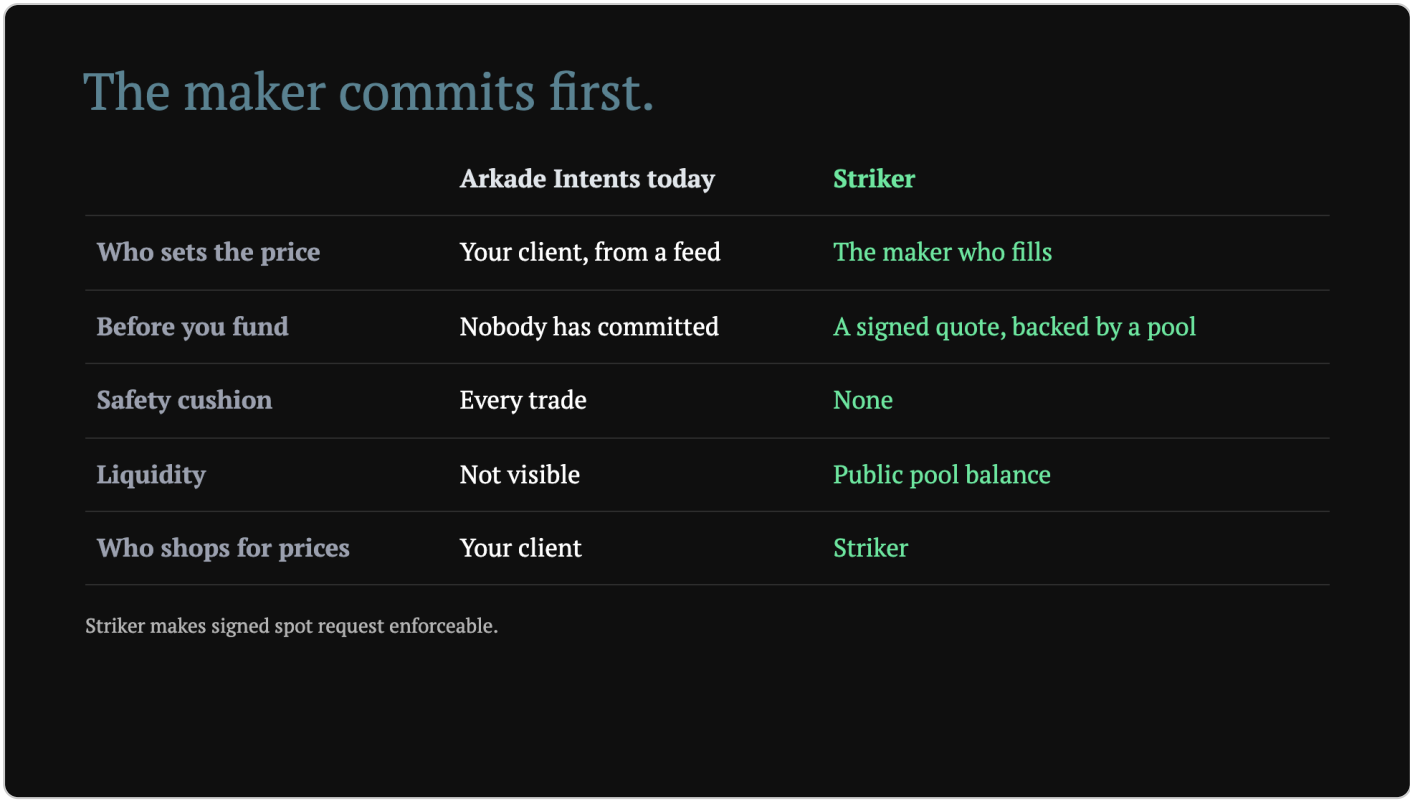
A covenant is a spending rule attached to a coin. It checks what a transaction does, as well as who authorized it.

In this implementation, the Arkade Emulator validates the program as part of the cooperative spending process.

The pool checks four things here: the maker signed the quote; it hasn't expired; the outputs satisfy the agreed payments; and the remaining liquidity returns to the same pool script.

That last rule lets one deposit support many trades. Each fill creates the next pool coin.

The user's covenant separately protects their receipt. Striker can assemble the transaction, but it cannot rewrite the terms those checks enforce.



07:40–08:30 · 50 seconds

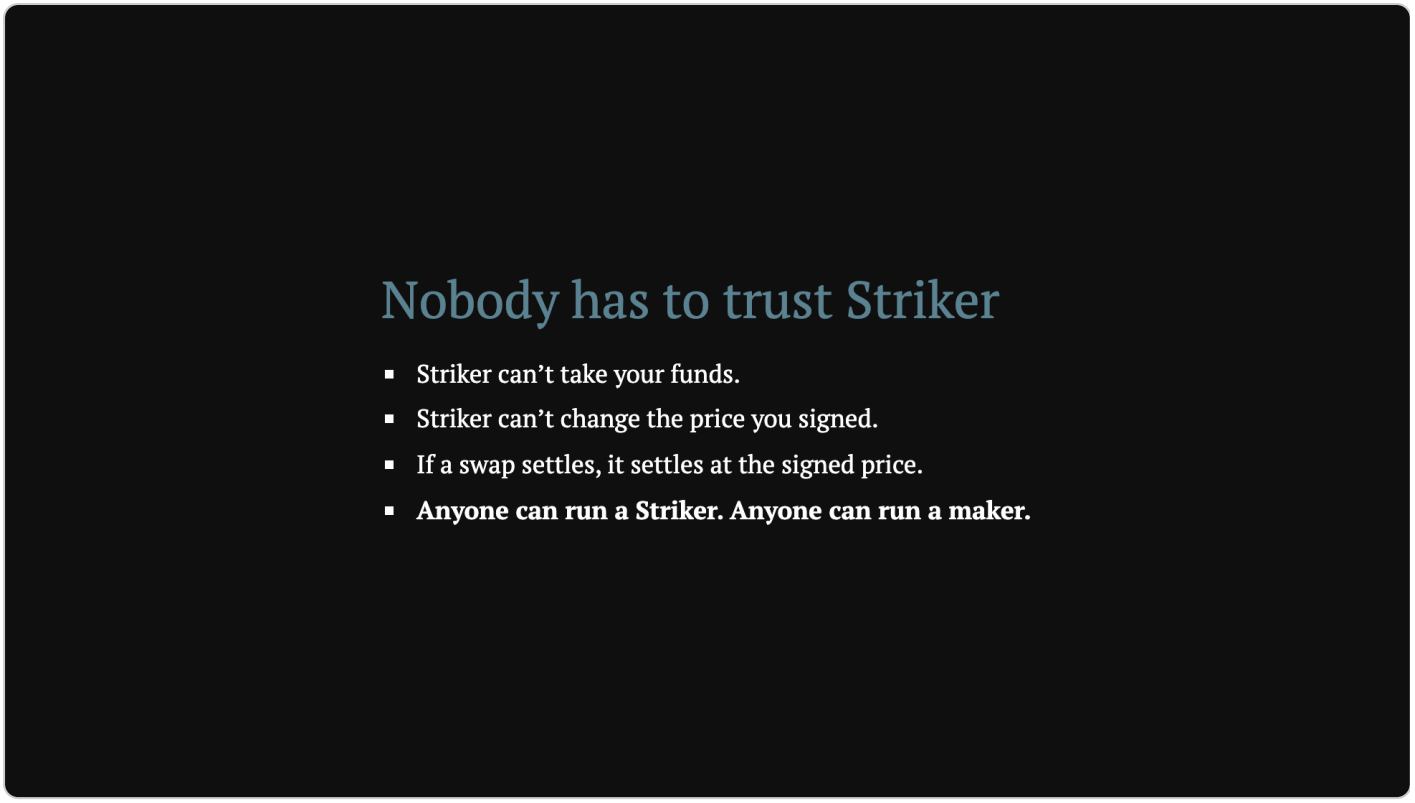
The maker commits first. That's the comparison I want you to remember.

The wallet no longer has to derive this offer from a feed and guess a cushion to make it attractive. The maker supplies the actual executable amounts. Its spread can still be included in that price.

New prices only require new signatures. Existing offers remain usable until expiry, unless the pool changes in a way that invalidates them.

For cancellation, an epoch is a pool version number. An owner-authorized spend can recreate the pool with a new epoch, so old offers no longer match.

That requires Arkade server cooperation, and it can race a fill. It isn't an instant off-chain revoke button.



08:30–09:30 · 60 seconds

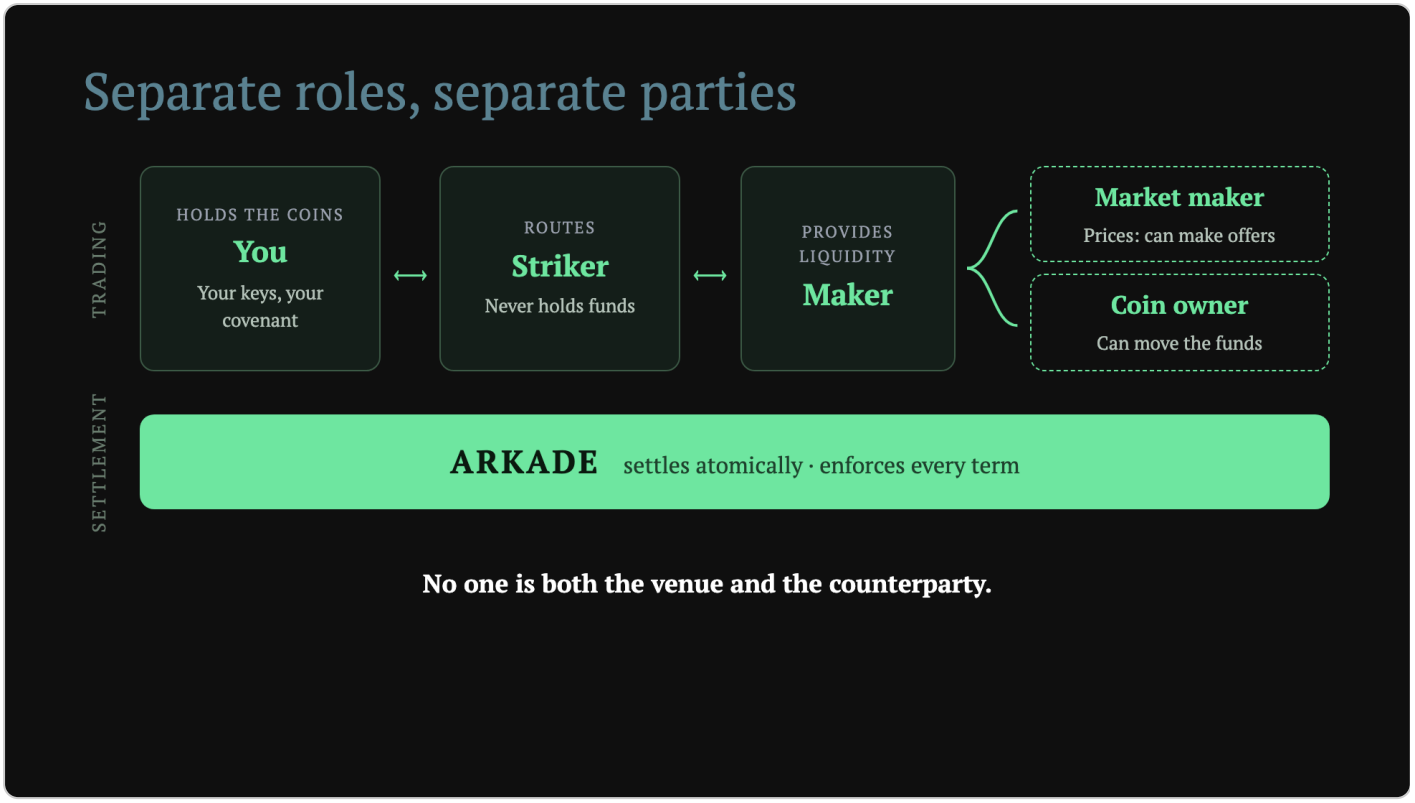
This slide says nobody has to trust Striker. Let me be precise about what that means.

You don't give Striker your private key or permission to choose a different price. Your wallet verifies the offer and authorizes specific terms. The spending rules constrain the resulting payments.

Striker can still refuse a request or delay it. A maker can withdraw liquidity or cancel outstanding quotes. A quote isn't a reservation of exclusive capacity.

And this proof of concept still depends on the Arkade server and Emulator. Recovery and cancellation currently require server cooperation; I'm not claiming unilateral exits for these contracts.

The useful property is narrower: the coordinator doesn't need custody to coordinate the trade. Another operator can run the same open-source service.



09:30–11:00 · 90 seconds

Now look at the roles rather than the machines.

The user owns their coins. Striker routes requests and builds settlements. The maker supplies liquidity and pricing. Arkade is the settlement environment underneath.

Those roles can be run by independent parties. They aren't required to be the same company. But independence is a deployment choice, not something the protocol can guarantee about the people running it.

The split drawn inside the maker is a further design direction: one party owns the liquidity, while another gets limited authority to sign quotes.

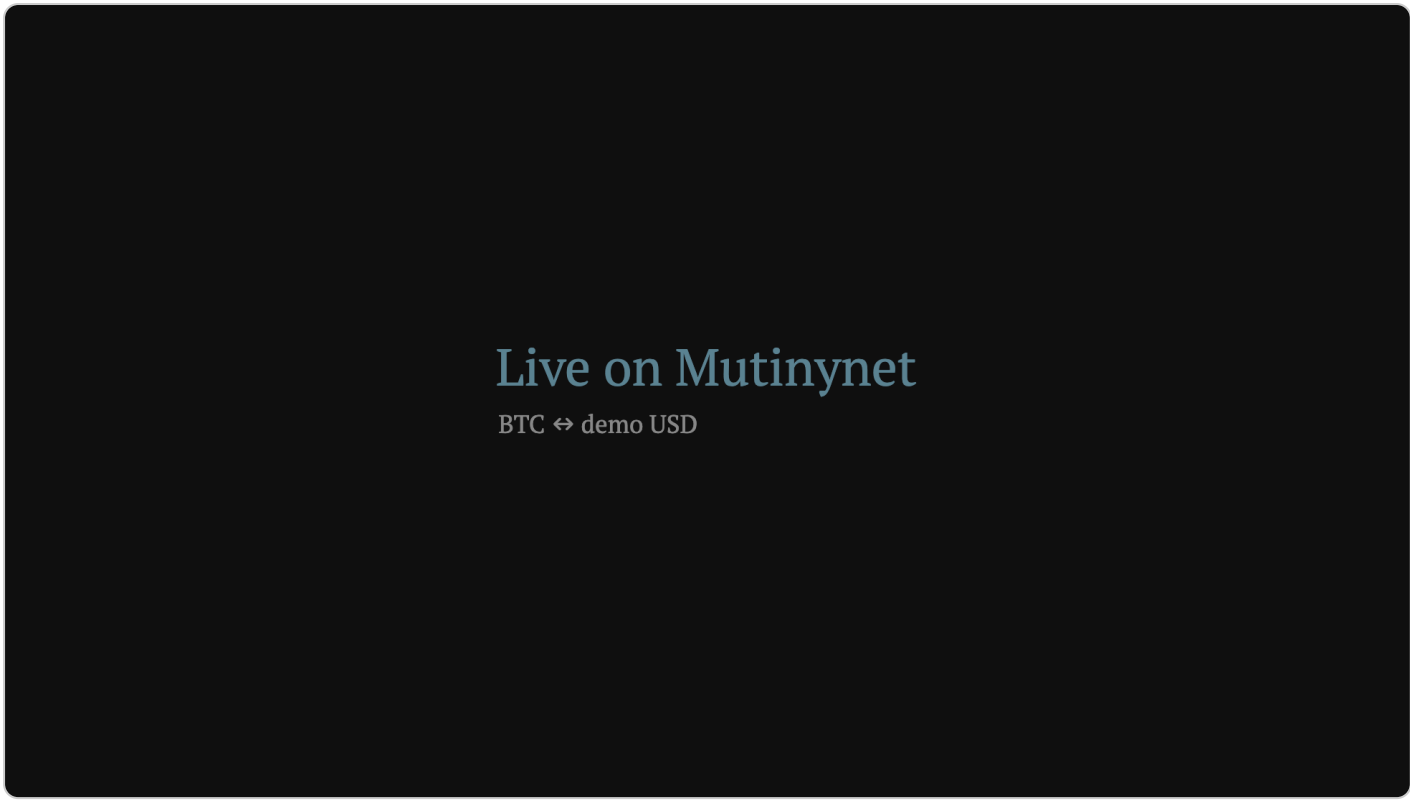
Think of an exchange API key with trading enabled and withdrawals disabled, except the restrictions would be expressed in the spending rules.

There's an important catch. A compromised quote key could still authorize a terrible price. Preventing arbitrary withdrawal doesn't prevent economic loss through bad trades.

So safe delegation also needs owner-defined limits, monitoring and a way to revoke authority.

Our deployed demo currently uses the same maker key for quoting and ownership. Separate keys, price floors and per-fill caps are work to build, not protections I'm claiming we already have.

The opportunity is that a signature is a much smaller job to delegate than unrestricted control of a wallet.



11:00–12:00 · 60 seconds

Let's look at the working application.

This is Mutinynet, and USD here is a demo token with eight decimals. It isn't backed dollars or genuine USDt.

[Enter a small amount and show the quote.]

The maker uses a live BTC–USDT feed to calculate these amounts, then signs its offer. You can see the expiry alongside the price.

[Review the swap.]

These are the terms the wallet verifies and authorizes.

[Execute, then show the confirmed result or the prepared recording.]

The receipt shows the outcome: both sides were paid in the same Arkade settlement.

The implementation is a testnet proof of concept. The point of the demo is that this coordination pattern already works for a real swap.

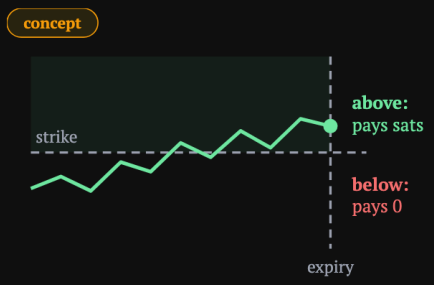


12:00–12:05 · 5 seconds

That's the working swap. Now, what could we build next?

Signed quotes open up more markets

Market	What the quote carries
Binary options	Strike, expiry, payout
Prediction markets	Event, odds
AMM-style pools	Price from a curve the maker computes
Futures	Mark price, margin terms



12:05–13:30 · 85 seconds

Signed terms over covenant-held collateral suggest more applications than spot swaps.

Take a fully collateralized binary option, paid in sats. Agree on the condition, expiry and payout, lock the required collateral, and resolve against an agreed price attestation.

That needs a new contract and an oracle design. Our moving chart is only a paper-trading UX experiment.

Prediction markets add event resolution. Futures add margin and liquidation. A maker can compute quotes from an AMM-style curve, but an autonomous AMM also needs its invariant enforced by the contract.

These are invitations to build on Arkade, not features I'm saying this swap script already implements.



13:30–15:00 · 90 seconds (includes buffer)

Close with one call to action; repeat the takeaway.

- "With Striker, the maker commits first, and the price lives in a signature. It all stands on Arkade, and there's room for a lot more on top."
- Ask: market makers, run a maker; wallet builders, integrate quotes; contract designers, pick a market from the table and build it.
- QR codes: left is the app, right is the repo. Thank Ark Labs by name.

Stop talking. Go to Q&A.